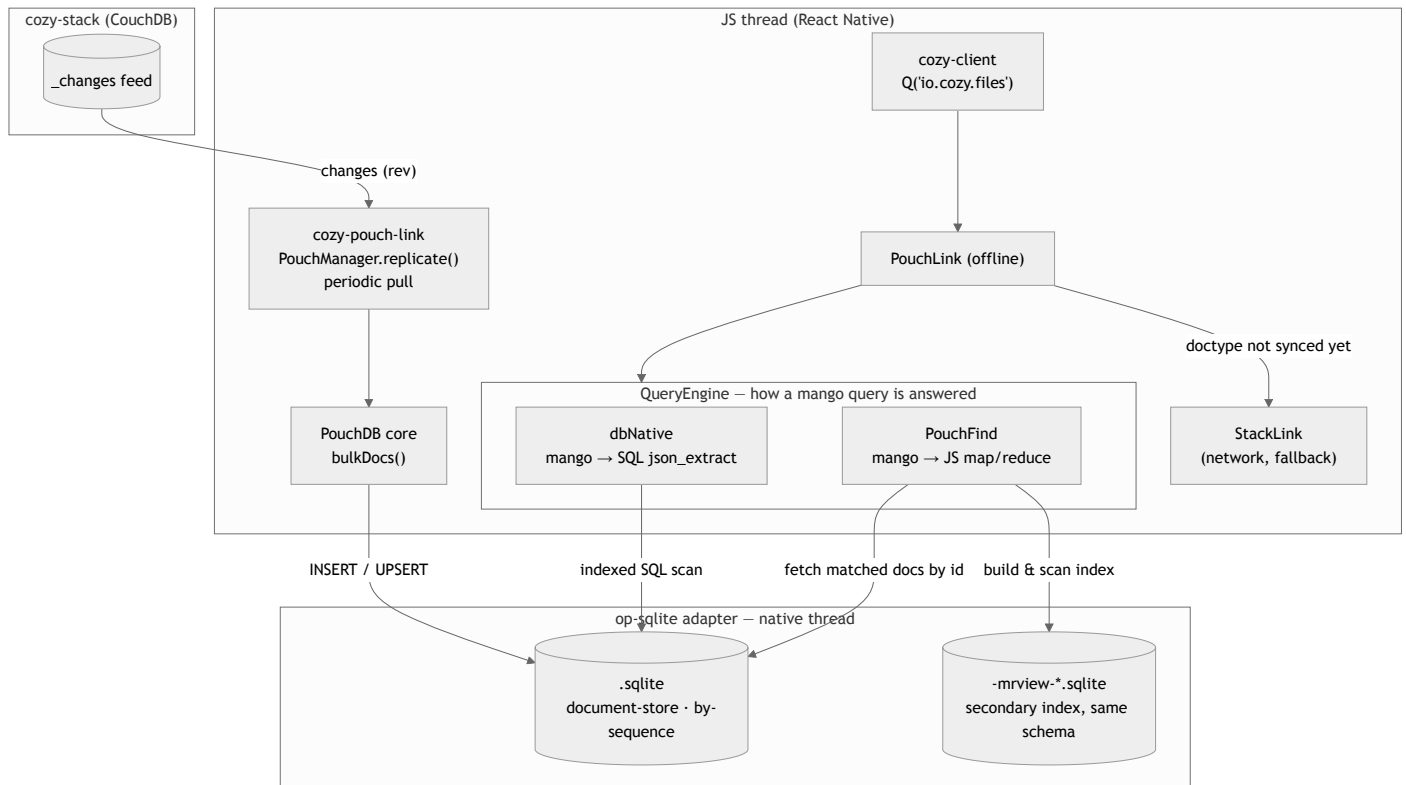


Pouch ↔ SQLite Sync

Twake Drive Mobile — offline-first · cozy-client + cozy-pouch-link + op-sqlite

The app is offline-first: all data lives in a local **SQLite** database and every query is answered from it. Two independent halves keep that database useful — a **sync (writes)** that pulls changes from cozy-stack (CouchDB) into SQLite, and **queries (reads)** that cozy-client runs against SQLite. They never talk to each other directly; they meet only in the database file.

1. The big picture



The adapter is always op-sqlite. Whichever QueryEngine is used, the data lives in `<db>.sqlite` and every read ends up there through op-sqlite. The QueryEngine does not decide *whether* SQLite is touched — it decides *how* the mango query is turned into work on top of it.

2. Writes — where the sync puts the bytes

The periodic pull reads the changed revisions from CouchDB's `_changes` feed and hands them to `PouchDB.bulkDocs()`. The adapter writes them into **two tables** of the main file `<db>.sqlite`:

by-sequence				document-store			
seq	json	doc_id	rev	id	winningseq	max_seq	json
1	{...}	fileA	1-a	fileA	2	2	{...}
2	{...}	fileA	2-b	←			
3	{...}	fileB	1-c	←	fileB	3	{...}

▲ one row PER revision (history is kept) ▲ one row PER document points at the winning revision

by-sequence — the append-only log

Every revision received (`seq` = `AUTOINCREMENT` primary key) becomes **a new row**. The revision history is never overwritten — that is what makes conflict resolution possible.

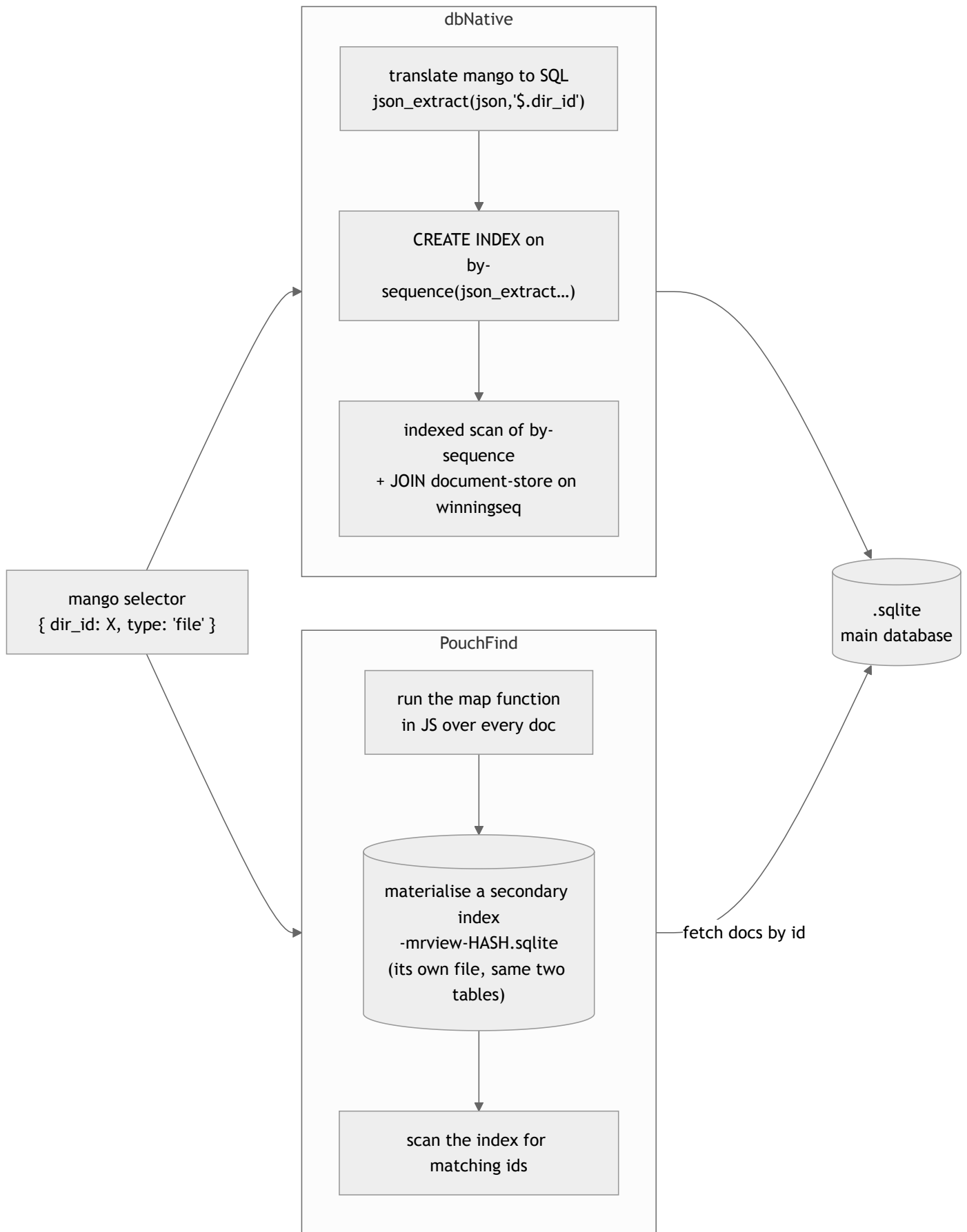
document-store — the "current state" index

One row per `doc_id`, whose `winningseq` points at the `by-sequence` row that holds the **winning revision**.

So applying one change = an `INSERT` into `by-sequence` plus an `UPSERT` of `document-store.winningseq/max_seq`. Everything goes through a single **serial queue**: reads and writes never run at the same time on the connection, and `op-sqlite` runs the SQL **off the JS thread**, so the UI is not blocked by the SQL itself.

3. Reads — one database, two QueryEngines

A query starts as a **mango selector** from cozy-client (e.g. `Q('io.cozy.files').where({ dir_id })`). It goes down the link chain: **PouchLink** answers from the local database; if that doctype has not been replicated yet, it falls through to **StackLink** (the network). Once the data is local, the query is answered by the **QueryEngine** — and there are two implementations of it. **Both read from SQLite through op-sqlite**; they differ only in how they turn the mango selector into work.



PouchFind

The default engine. A mango index is registered as a PouchDB *design document*; PouchDB's map/reduce layer then **runs the map function in JavaScript over every document** and **materialises a secondary-index database** — its own `<db>-mrview-<hash>.sqlite` file, with the same `document-store / by-sequence` schema, written through the same `op-sqlite`

adapter. A query then range-scans that index for the matching document ids and **fetches the documents from the main database**. Everything is SQLite under op-sqlite — but building the index the first time is a **full pass over every document in JS**, which is the cost paid on the first query.

dbNative

Opens a direct connection to the **same** `<db>.sqlite` and **translates the mango selector into SQL** against the `json` column (`json_extract`), backed by a real SQLite index on the extracted field. There is no JS map function and no separate index file — the query runs straight against `by-sequence` . To return only the **winning revision** (and not the whole revision history that lives in `by-sequence`), it joins `document-store` :

```
SELECT bs.json AS data, bs.doc_id, bs.rev
FROM 'by-sequence' bs
JOIN 'document-store' ds ON bs.seq = ds.winningseq  -- winning rev only
WHERE json_extract(bs.json, '$.dir_id') = 'X' AND bs.deleted = 0;
```

Same data, two ways to find it

	PouchFind	dbNative
Where the answer lives	secondary-index db + main db	main db only
How the index is built	map function run in JS over every doc	SQLite index on <code>json_extract</code>
Extra SQLite files	one <code>-mrview-*</code> per index	none
First-query cost	full JS pass to build the view	one <code>CREATE INDEX</code>
Adapter underneath	op-sqlite	op-sqlite

In one sentence: the sync writes each revision into `by-sequence` and updates the `winningseq` pointer in `document-store` ; a read then finds those rows either by scanning a map/reduce index that PouchFind materialises in a separate SQLite file, or by running indexed `json_extract` SQL straight against the main database with dbNative — both landing in SQLite through op-sqlite.